Flow Matching Policy Gradients

> #RL

Algorithm 1 Flow Policy Optimization (FPO)

Require: Policy parameters θ, value function parameters ϕ, clip parameter ϵ, MC samples N_{mc}

```
while not converged do
    Collect trajectories using any flow model sampler and compute advantages Â_t
    For each action, store N_mc timestep-noise pairs {(τ_i, ε_i)} and compute ℓ_θ(τ_i, ε_i)
    θ_old ← θ
    for each optimization epoch do
        Sample mini-batch from collected trajectories
        for each state-action pair (o_t, a_t) and corresponding MC samples {(τ_i, ε_i)} do
            Compute ℓ_θ(τ_i, ε_i) using stored (τ_i, ε_i)
            r̂_θ ← exp( − (1/N_mc) Σ_{i=1}^{N_mc} (ℓ_θ(τ_i, ε_i) − ℓ_{θ_old}(τ_i, ε_i)) )
            L^FPO(θ) ← min(r̂_θ Â_t, clip(r̂_θ, 1 ± ε) Â_t)
        end for
        θ ← Optimizer(θ, ∇_θ Σ L^FPO(θ))
    end for
    Update value function parameters φ like standard PPO
end while
```

- What problem does this framework solve?
  - PPO is a classic and effective way of finetuning a model. It relies on the ratio of distributions: $r(\theta) = \frac{\pi_\theta(a_t|o_t)}{\pi_{old}(a_t|o_t)}$.
  - This ratio is easy to compute for Gaussian policies. However, in multimodal tasks and exact log-likelihoods, Gaussian policies are not enough.
  - Flow models are good at handling these tasks; however, the exact log-likelihoods needed to compute r is hard to obtain for flow models.
    - This is because when we find the exact log-likelihoods of flow model, we need to find the integral of the divergence of an ODE. Specifically, for $\frac{dx(t)}{dt} = v_\theta(t)$, we need to find: $\log p_1(x_1) = \log p_0(x_0) - \int_0^1 \operatorname{div}(v_\theta(x_t, t))dt$, where $\operatorname{div}(v_\theta) = \operatorname{Tr}\left(\frac{\partial v_\theta(x_t,t)}{\partial x_t}\right) = \sum_{i=1}^{D} \frac{\partial v_i}{\partial x_i}$. Here, $v_\theta(\cdot)$ and x_t are all high-dimensional vectors; plus the integral, it is nearly impossible to calculate the log.
- What does this framework do?
  - Instead of computing the exact log-likelihood, the paper proposes a way to convert it to the difference of CFM loss (the loss of flow match model).
  - We keep the objective function of PPO unchanged: $\max_\theta \mathbb{E}_{a_t \sim \pi_\theta(a_t|o_t)}[\min(\hat{r}^{FPO}(\theta)\hat{A}_t, clip(\hat{r}^{FPO}(\theta), 1-\epsilon^{clip}, 1+\epsilon^{clip})\hat{A}_t)]$, but replace the r with $\hat{r}^{FPO}(\theta) = \exp(\hat{\mathcal{L}}_{CFM,\theta_{old}}(a_t; o_t) - \hat{\mathcal{L}}_{CFM,\theta}(a_t; o_t))$. *How:*
  - Notice that we can represent the log-likelihood as:

$$\log \pi_\theta(a_t|o_t) = \underbrace{\mathbb{E}_{q(z|a_t)}\left[\log \frac{p_\theta(a_t,z|o_t)}{q(z|a_t)}\right]}_{\text{This is } ELBO_\theta(a_t|o_t)} + \underbrace{\mathbb{E}_{q(z|a_t)}\left[\log \frac{q(z|a_t)}{p_\theta(z|a_t,o_t)}\right]}_{\text{This is the KL}}$$

This provides us with a way to estimate the log-likelihood: we use the ELBO, where the only difference is the KL (**so called KL-gap**). Then, we have:

$$ELBO_\theta(a_t|o_t) = \log\ \pi_\theta(a_t|o_t) - \mathcal{D}_\theta^{KL}.$$

- To better decouple the KL and the log-likelihood, we use the ratio of the exponential of ELBO:

  $$r^{FPO}(\theta) = \frac{\exp(ELBO_\theta(a_t|o_t))}{\exp(ELBO_{\theta_{old}}(a_t|o_t))}$$

By log's property and exponential's property, we have:

$$r^{FPO}(\theta) = \frac{\pi_\theta(a_t|o_t)}{\pi_{\theta_{old}}(a_t|o_t)} \frac{\exp(\mathcal{D}_{\theta_{old}}^{KL})}{\exp(\mathcal{D}_\theta^{KL})}$$

- Then traditional PPO is to maximize this ratio. This provides 2 effects:
  1. According to the definition of ELBO, maximize ELBO = maximize the probability of $\log \pi_\theta(a_t|o_t)$, which is what we want.
  2. Minimize the KL gap: $\frac{\exp(\mathcal{D}_{\theta_{old}}^{KL})}{\exp(\mathcal{D}_\theta^{KL})}$
- According to Kingma & Gao's work, we can represent the ELBO as:

  $\mathcal{L}_\theta^w(a_t) = -ELBO_\theta(a_t) + c$, where

  $$\mathcal{L}_\theta^w(a_t;o_t) = \frac{1}{N_{mc}} \sum_{i=1}^{N_{mc}} \underbrace{\left|\left|\hat{v}_\theta(a_t^{\tau_i}, \tau_i; o_t) - (a_t - \epsilon_i)\right|\right|_2^2}_{\text{MSE}}$$

  This is the objective function of training flow matching models. We want to predict a change in each element to recover the original image. $\hat{v}_\theta$ is the model's output (a flow, or, a "speed"), and $a_t - \epsilon_i$ is the groundtruth change (here ϵ_i, the noise, is the start, and a_t is the end, so the speed = end - start = $a_t - \epsilon_i$). We use Monte Carlo sampling + MSE as the loss function.
- With this, we have:

  $$r_\theta^{FPO} = \frac{\exp(ELBO_\theta(a_t|o_t))}{\exp(ELBO_{\theta_{old}}(a_t|o_t))} = \exp(\mathcal{L}_{\theta_{old}}^w(a_t) - \mathcal{L}_\theta^w(a_t))$$

- Experiment Results:
  - GridWorld: In “saddle point” states where multiple equivalent goals exist, FPO successfully learned a multimodal action distribution (where moving left or right can both reach the goal), whereas traditional Gaussian policies only output trajectories leading to a single goal, lacking exploration and diversity.
  - MuJoCo Playground: In 10 standard continuous control tasks, FPO was trained from scratch and outperformed the standard Gaussian PPO as well as the previous MDP-based diffusion reinforcement learning method, DPPO, in 8 of these tasks.
  - Humanoid Control: In the high-dimensional Isaac Gym physics simulation, FPO performs comparably to PPO when conditions are sufficient (i.e., all joint information is provided); however, in under-conditioned scenarios (e.g., where only root node or hand movement commands are provided), the Gaussian strategy fails to learn viable walking actions and frequently falls, whereas FPO maintains a high success rate and smoothly tracks the target, demonstrating robust distribution modeling capabilities.
- Work Cited
  - Paper

- Chat History